

Day1. Gradle 빌드 및 프로젝트 실습

1. 코드 실습

```
// 프로젝트에서 사용하고자 하는 외부 라이브러리를 명시
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;
```

```
// Map을 생성한다.
// Map: <키, 값>으로 관리하는 자료구조.
// 해당 프로젝트에서는 키를 ID, 값을 PW로 설정.
private static final Map<String, String> db = new HashMap<>();
```

```
// 클래스에서 요소를 사용하기 위해서는 객체화를 해야 사용가능.
// static이 붙어있는 것은 이러한 객체화 과정없이 사용가능하게 함.
// static 코드 블록에는 프로그램 실행시 자동으로 동작되는 부분을 넣음.
static {
    // 위에서 만든 db에 데이터를 넣음.
    db.put("alice", "pass1234");
    db.put("bob", "mySecret");
}
```

```
// new 키워드를 통해 객체를 생성함.
// Scanner는 키보드 입력을 받는 객체. 라인 단위로 입력을 받음.
Scanner sc = new Scanner(System.in);
System.out.println("=== Day 1: 간이 로그인 시스템 ===");
```

```
// 사용자의 로그인 정보를 입력받음.
System.out.print("아이디: ");
// nextLine() : 키보드로 한 줄 입력 받음.
// trim() : 좌우 공백 제거
String id = sc.nextLine().trim();

System.out.print("비밀번호: ");
String pw = sc.nextLine().trim();
```

```
// Map에서 제공하는 함수 중 containsKey는 사용자가 입력한 ID 값이 db안에 있는지 검사
// 만약 있다면 true, 없다면 false를 반환
// false를 반환한 경우 Not 연산자를 통해 if문 블록을 실행.
if (!db.containsKey(id)) {
    // 출력
    System.out.println("존재하지 않는 ID입니다.");
    // main 종료 -> 프로그램 종료
    return;
}
```

```
// Map에서 키를 통해 값을 가져오기 위해서는 get() 함수를 사용.
// 이를 통해 이전에 설정한 db의 값 타입인 문자열을 반환
// JAVA에서는 문자열 비교를 equals() 함수를 통해 해야 함.
// == 비교시 제대로 된 결과를 반환하지 않음.
if (db.get(id).equals(pw)) {
    System.out.println("로그인 성공! 환영합니다");
} else {
```

```
        System.out.println("비밀번호가 틀렸습니다");  
    }  
}
```

2. jar 빌드과정

1. gradle 프로젝트 생성
 2. 프로젝트 코드 작성 후, 터미널 실행
 3. 루트 폴더 확인 후, `./gradlew clean build` 명령어 실행.
 1. 만약, gradle을 찾을 수 없다고 오류가 발생 시: 폴더 트리에서 gradlew 폴더가 있는지 확인
 4. 이후 `app/build/libs` 경로 아래에 *.jar 파일이 생성되었는지 확인
 5. `java -cp app/build/libs/app.jar login_server.App` 명령어를 통해 테스트
-

3. 프로그램의 문제점

- 보안상의 문제: 비밀번호 저장을 평문으로 처리하고 있습니다. 암호화가 필요합니다.
- 데이터 관리의 문제: 메모리에 저장되는 데이터의 양은 한계가 있기 때문에, 대용량의 데이터를 저장하지 못합니다.